Data Mining Exam Answers

https://gemini.google.com/app/35014a4ad1337472

User prompt: give me a Short and easy exam ready answer of this question according to the given marks

Response: 1. a) What is data type in Data Mining? What types of data are found in Data Mining? Name those data types. [3 Marks]

Definition:

In data mining, a **data type** (or attribute type) defines the nature, structure, and domain of values that an attribute or variable can hold. It determines what kind of statistical operations and mining algorithms can be applied to the data.

Types of Data Found in Data Mining:

1. **Qualitative (Categorical) Data:**
    - **Nominal:** Values are symbols or names without any natural order (e.g., *Color: Red, Blue, Green; Marital Status*).
    - **Binary:** Nominal attributes with only two states (e.g., *Yes/No, 0/1, True/False*).
    - **Ordinal:** Values have a meaningful order or ranking, but the difference between values is not quantifiable (e.g., *Customer Rating: Low, Medium, High; Grades: A, B, C*).
2. **Quantitative (Numeric) Data:**
    - **Interval-scaled:** Values have an order and equal intervals, but no true absolute zero point (e.g., *Temperature in °C or °F, Calendar Dates*).
    - **Ratio-scaled:** Numeric values with an absolute zero point where ratios are meaningful (e.g., *Age, Salary, Weight*).

1. b) How do you collect and clean data? Is there any specific way to accomplish this work? [5 Marks]

1. Data Collection

Data collection involves gathering raw data from multiple internal and external sources:

- **Databases & Data Warehouses:** SQL queries, enterprise ERP/CRM systems.
- **Web Scraping & APIs:** Extracting data from online platforms, web services, or REST APIs.
- **IoT & Sensor Feeds:** Real-time data streams from physical devices.
- **Surveys & Logs:** Server access logs, event streams, or form submissions.

2. Data Cleaning

Raw data is often "dirty" (incomplete, noisy, or inconsistent). The cleaning process resolves these issues:

- **Handling Missing Data:** Dropping incomplete tuples or imputing missing attributes.
- **Smoothing Noisy Data:** Removing random errors or outliers using binning, clustering, or regression.
- **Resolving Inconsistencies:** Correcting discrepancies in naming formats, units, or duplicate records.

3. Specific Way / Standard Framework to Accomplish the Work

Yes, standard systematic methodologies guide this end-to-end process:

- **CRISP-DM (Cross-Industry Standard Process for Data Mining):** Follows defined phases: *Business Understanding* $\rightarrow$ *Data Understanding (Collection)* $\rightarrow$ *Data Preparation (Cleaning & Transformation)* $\rightarrow$ *Modeling* $\rightarrow$ *Evaluation* $\rightarrow$ *Deployment*.
- **ETL Pipeline (Extract, Transform, Load):** Used in data warehousing to programmatically extract raw sources, transform/clean the data via automated rules, and load it into a central repository for mining.

1. c) What are the techniques to fill in missing values? [6 Marks]

Missing values occur due to equipment malfunction, non-response in surveys, or transmission errors. The primary techniques to handle and fill them include:

1. **Ignore the Tuple (Listwise Deletion):**

- Drops the entire row containing the missing value.
- *Best when:* The dataset is large and only a tiny percentage of rows have missing values (especially when the class label itself is missing).

2. **Fill in Manually:**
   - An expert inputs the value manually based on domain knowledge.
   - *Limitation:* Highly time-consuming and unfeasible for large datasets.
3. **Use a Global Constant:**
   - Replaces all missing values with a single constant such as `"Unknown"`, `"N/A"`, or $-\infty$.
   - *Limitation:* May create artificial patterns that mislead learning algorithms.
4. **Use a Measure of Central Tendency (Mean or Median):**
   - **Symmetric Data**: Replace missing values with the **Mean** of the attribute.
   - **Skewed Data:** Replace missing values with the **Median** to avoid distortion from outliers.
5. **Use Attribute Mean/Median for the Same Class:**
   - Segment the data by class label (e.g., customers of the same risk level) and use the mean or median of that specific class to fill the missing value. This provides higher accuracy than a global mean.
6. **Predictive Modeling / Advanced Imputation**:
   - Treat the missing attribute as the target variable and predict it using other attributes:
     - **Regression:** Fits a regression line/curve to estimate numeric values.
     - **K-Nearest Neighbors (KNN Imputation):** Finds the k most similar records based on available features and averages their values.
     - **Decision Trees / Random Forests:** Predicts categorical or numerical values using rule-based branching.
   - *Advantage:* Retains complex relationships between attributes better than simple statistical substitution.

Response: 2. a) It is known that learning is divided into two major classes: Supervised learning and Unsupervised learning. How these learning types influence data mining works? [4 Marks]

Supervised and unsupervised learning form the foundational paradigms of data mining, dictating the objectives, preparation, algorithms, and evaluation methods used throughout the mining process:

- **Supervised Learning Influence:**
  - **Role & Goal:** Applied when the target/outcome variable (class label) is known. The goal is to build a model that maps input features to a predetermined output.
  - **Influence on Mining Tasks:** Drives **Classification** (predicting discrete labels like *Fraud / Non-Fraud*) and **Regression** (predicting continuous numerical values like *House Prices*).
  - **Workflow Impact:** Requires pre-labeled training data; uses metrics like accuracy, precision, recall, and Mean Squared Error (MSE) to validate models.
- **Unsupervised Learning Influence:**
  - **Role & Goal:** Applied when the data contains no predefined target labels. The goal is to discover underlying patterns, natural groupings, and internal structures directly from the data.
  - **Influence on Mining Tasks:** Drives **Clustering** (grouping similar records, e.g., customer segmentation via K-Means), **Association Rule Mining** (discovering co-occurring items, e.g., market basket analysis using Apriori), and **Dimensionality Reduction / Outlier Detection** (e.g., PCA, anomaly detection).
  - **Workflow Impact:** Eliminates the need for manual labeling; evaluation relies on intrinsic metrics like silhouette scores, support, and confidence.

2. b) What are the Data Mining programming languages used in the current time? What are the common data repositories available? [4 Marks]

1. Programming Languages Used for Data Mining:

- **Python:** The industry standard due to extensive data libraries (Pandas, NumPy, Scikit-learn, PyTorch, TensorFlow).
- **R:** Heavily utilized for advanced statistical modeling, data analysis, and graphical representations (ggplot2, caret).
- **SQL:** Essential for querying, extracting, aggregating, and manipulating structured tabular data in relational databases.
- **Scala / Java:** Used for large-scale distributed big data processing and mining frameworks like Apache Spark and Hadoop.
- **Julia:** Gaining popularity for high-performance scientific and numerical computing.

2. Common Data Repositories Available:

- **UCI Machine Learning Repository:** Classic academic repository providing diverse benchmark datasets for classification, regression, and clustering.
- **Kaggle Datasets:** Community-driven platform offering thousands of real-world datasets across multiple domains.
- **Government Open Data Portals:** Public domain repositories providing official demographic, economic, and civic data (e.g., *Data.gov*, *data.gov.uk*).
- **Google Dataset Search / Hugging Face Datasets:** Centralized search tools and hubs for locating structured, NLP, and multimodal datasets.
- **AWS Public Datasets / Google Cloud Public Datasets:** Cloud-hosted repositories providing large-scale scientific, satellite, and economic data ready for cloud-based mining.

2. c) Write about Information Gain and Gini Index. [6 Marks]

Both **Information Gain** and **Gini Index** are attribute selection measures (splitting criteria) used in Decision Tree algorithms to determine the best attribute for partitioning data at each node.

1. Information Gain (Used in ID3 and C4.5)

- **Concept:** Measures the expected reduction in **Entropy** (impurity/uncertainty) resulting from splitting the dataset on a given attribute A.
- **Entropy Formula:** For a dataset D with m classes:

$$\text{Entropy}(D) = -\sum_{i=1}^{m} p_i \log_2(p_i)$$

(where p_i is the probability that an arbitrary tuple in D belongs to class i.)

- **Information Gain Formula:**

$$\text{Gain}(D, A) = \text{Entropy}(D) - \sum_{j=1}^{v} \frac{|D_j|}{|D|} \text{Entropy}(D_j)$$

(where attribute A partitions D into v subsets.)

- **Selection Rule:** The attribute with the **highest Information Gain** is chosen as the splitting attribute.
- **Limitation:** Strongly biased toward attributes with many distinct values (e.g., `CustomerID` or `TransactionID`).

2. Gini Index (Used in CART)

- **Concept:** Measures the degree of inequality or impurity among the distributions of elements in a dataset. It quantifies the probability of a randomly chosen element being incorrectly labeled if it were randomly labeled according to the distribution of labels in the subset.
- **Gini Index Formula:** For a dataset D:

$$\text{Gini}(D) = 1 - \sum_{i=1}^{m} p_i^2$$

- **Split Evaluation Formula:** When a binary split on attribute A divides D into D_1 and D_2:

$$\text{Gini}_A(D) = \frac{|D_1|}{|D|}\text{Gini}(D_1) + \frac{|D_2|}{|D|}\text{Gini}(D_2)$$

- **Reduction in Impurity:**

$$\Delta\text{Gini}(A) = \text{Gini}(D) - \text{Gini}_A(D)$$

- **Selection Rule:** The attribute that yields the **minimum** $\text{Gini}_A(D)$ (or maximum $\Delta\text{Gini}(A)$) is selected as the best split.

Key Differences:

Feature	Information Gain	Gini Index
Primary Algorithm	ID3, C4.5	CART (Classification and Regression Trees)
Underlying Metric	Based on Entropy ($\log_2$)	Based on squared probabilities (p_i^2)
Computation Speed	Slower (requires logarithmic calculations)	Faster (uses simple arithmetic squares)
Tree Structure	Allows multiway splits	Strictly produces binary splits

---

Response: 3. a) Why are test and training data separated? What is the benefit of doing so? [5 Marks]

Why They Are Separated:

- **Simulating Real-World Deployment:** The primary objective of a machine learning model is to make accurate predictions on new, unseen data. Evaluating the model on the same data it learned from yields an overly optimistic, biased performance metric.
- **Preventing Data Leakage:** Separating data ensures that the training phase has zero access to the ground truth of the test set, maintaining strict experimental integrity.

Benefits of Separation:

1. **Detecting and Preventing Overfitting:** Overfitting occurs when a model memorizes the training data, including noise and outliers, rather than learning general patterns. A separate test set exposes whether the model has overfit.
2. **Unbiased Generalization Estimate:** Testing on held-out data provides an objective measurement of how well the model generalizes to real-world scenarios.
3. **Objective Model Comparison & Selection:** A standardized test set provides a fair, identical benchmark to compare different algorithms or hyperparameter configurations.

3. b) Why do we need cross validation in Data Mining operations? [3 Marks]

Cross-validation (e.g., k-Fold Cross-Validation) is essential in data mining operations for the following reasons:

1. **Overcoming Data Scarcity:** In smaller datasets, setting aside a large fixed test split leaves insufficient data for training. Cross-validation ensures every data point gets used for both training and validation across different iterations.
2. **Mitigating Sampling Bias:** A single train/test split can produce misleading results if an easy or unusually difficult subset ends up in the test set by chance. Cross-validation averages results across multiple folds, reducing variance.
3. **Reliable Hyperparameter Tuning:** It provides a dependable validation score during feature selection and model parameter optimization without touching or compromising the final test set.

3. c) “Leave-one-out cross-validation is a good way to validate any data” – Do you agree or disagree? Why? [6 Marks]

Verdict:

I disagree. While Leave-One-Out Cross-Validation (LOOCV) is useful in specific scenarios, it is **not** universally suitable for "any data."

What is LOOCV?

LOOCV is an extreme variant of k-fold cross-validation where $k = N$ (the total number of records). The model is trained N times, each time using $N - 1$ samples for training and exactly 1 sample for validation.

Why It Is NOT Good for Any Data (Reasons for Disagreement):

1. **Extreme Computational Cost:**
   - Training a model N times becomes practically impossible on medium-to-large datasets. For instance, training a complex model (like an ensemble or neural network) on a dataset with $100,000$ samples would require $100,000$ training cycles.
2. **High Variance of the Test Error:**
   - Because each validation set consists of only a single observation, the test error estimates fluctuate wildly between folds. Furthermore, the N training sets are almost identical (sharing $N-2$ data points), making the outputs of individual folds highly correlated. Averaging highly correlated estimates leads to higher variance in overall performance assessment.
3. **Failure on Dependent / Time-Series Data:**
   - LOOCV assumes that instances are independent and identically distributed (i.i.d.). On time-series, sequential, or spatial data, predicting an omitted point using future points leads to data leakage and invalid performance metrics.
4. **Poor Fit for Imbalanced Data:**
   - In severely imbalanced datasets, holding out a single sample can distort class ratios, especially when evaluating minority classes. Stratified k-fold is far more reliable in preserving label proportions.

When LOOCV Is Actually Useful:

LOOCV is strictly advantageous for **very small datasets** (e.g., $N < 100$ in bioinformatics or rare medical trials) where every single data point is precious and models are computationally inexpensive to train. For general, large-scale, or real-world mining tasks, standard **5-fold or 10-fold cross-validation** is preferred.

Response: 4. a) Write a note on incremental Association Rule Mining. [3 Marks]

Incremental Association Rule Mining refers to the process of maintaining and updating discovered association rules when new transactions are added to an existing database, without mining the entire cumulative database from scratch.

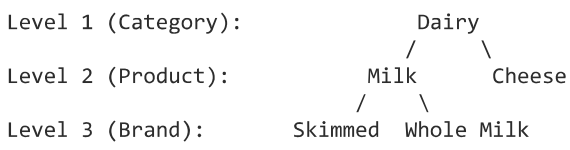
- **Need/Motivation:** Real-world transaction databases grow continuously over time. Re-running traditional mining algorithms (like standard Apriori) over the entire database whenever updates occur is computationally prohibitive.
- **Core Idea:** It efficiently combines the previously mined frequent itemsets and rules with the newly added batch of data (ΔD). An itemset might remain frequent, become newly frequent, or drop out of significance.
- **Key Approaches:** Algorithms like **FUP (Fast Update Algorithm)** maintain itemset counts and examine only candidate itemsets that could become frequent in the combined dataset, drastically reducing the number of database scans.

4. b) Explain Mining Multiple-Level Association Rules using example. [4 Marks]

Concept:

In real-world applications, items often belong to taxonomies or concept hierarchies (e.g., *Category* $\rightarrow$ *Sub-category* $\rightarrow$ *Specific Brand/Model*). **Multiple-Level Association Rule Mining** discovers rules at different abstraction levels within a concept hierarchy:

- Rules at higher abstraction levels typically have higher support but may be too generic.
- Rules at lower levels provide specific, actionable insights but have lower support.



Example:

- **Higher-level rule (Level 1):**

$$\text{Dairy} \implies \text{Bread} \quad [\text{Support} = 15\%, \text{Confidence} = 80\%]$$

- **Lower-level rule (Level 2/3):**

$$\text{Whole Milk} \implies \text{Wheat Bread} \quad [\text{Support} = 3\%, \text{Confidence} = 85\%]$$

To handle the sparsity of support at deeper levels, a **reduced minimum support strategy** is commonly used, where lower minimum support thresholds are applied at deeper levels of the hierarchy.

4. c) State the Apriori Property. Generate large itemsets and association rules using Apriori algorithm on the following data set with minimum support value and minimum confidence value set as 50% and 75% respectively. [7 Marks]

1. Apriori Property

> **Apriori Property (Downward Closure Property):**
>
> All nonempty subsets of a frequent (large) itemset must also be frequent. Conversely, if an itemset is infrequent, all of its supersets are guaranteed to be infrequent and are pruned immediately.

2. Given Data & Thresholds

- Total number of transactions: $N = 4$
- Minimum Support (min_sup) = $50\% \implies \text{min_count} = 4 \times 0.50 = 2$
- Minimum Confidence (min_conf) = $75\% = 0.75$

Let the item abbreviations be:

- **C:** Cheese
- **M:** Milk
- **K:** Cookies
- **B:** Butter
- **Br:** Bread

Transactions:

- **T101:** {C, M, K}
- **T102:** {B, M, Br}
- **T103:** {C, B, M, Br}
- **T104:** {B, Br}

3. Step-by-Step Itemset Generation

Step 1: Candidate 1-itemsets (C_1) & Frequent 1-itemsets (L_1)

Itemset	Support Count	Status (≥ 2)
{Cheese}	2	**Frequent**
{Milk}	3	**Frequent**
{Cookies}	1	*Pruned* (< 2)
{Butter}	3	**Frequent**
{Bread}	3	**Frequent**

L_1 **= { {Cheese}, {Milk}, {Butter}, {Bread} }**

Step 2: Candidate 2-itemsets (C_2) & Frequent 2-itemsets (L_2)

Join $L_1 \bowtie L_1$:

Itemset	Transactions	Support Count	Status (≥ 2)
{Cheese, Milk}	T101, T103	2	**Frequent**

Itemset	Transactions	Support Count	Status (≥ 2)
{Cheese, Butter}	T103	1	*Pruned*
{Cheese, Bread}	T103	1	*Pruned*
{Milk, Butter}	T102, T103	2	**Frequent**
{Milk, Bread}	T102, T103	2	**Frequent**
{Butter, Bread}	T102, T103, T104	3	**Frequent**

L_2 = { {Cheese, Milk}, {Milk, Butter}, {Milk, Bread}, {Butter, Bread} }

Step 3: Candidate 3-itemsets (C_3) & Frequent 3-itemsets (L_3)

Join $L_2 \bowtie L_2$:

- Candidate: {Milk, Butter, Bread}
- Check subsets: {Milk, Butter} $\in L_2$, {Milk, Bread} $\in L_2$, {Butter, Bread} $\in L_2$ (passes pruning).

Count support:

- {Milk, Butter, Bread} occurs in: **T102, T103** $\implies$ $\text{Support Count} = 2 \geq 2$.

L_3 = { {Milk, Butter, Bread} }

(No candidate 4-itemsets can be generated since at least two 3-itemsets are required for joining).

Final Large (Frequent) Itemsets:

- 1-itemsets: {Cheese}, {Milk}, {Butter}, {Bread}
- 2-itemsets: {Cheese, Milk}, {Milk, Butter}, {Milk, Bread}, {Butter, Bread}
- 3-itemsets: {Milk, Butter, Bread}

4. Association Rule Generation ($\text{min_conf} \geq 75\%$)

$$\text{Confidence}(X \implies Y) = \frac{\text{Support Count}(X \cup Y)}{\text{Support Count}(X)}$$

From L_2:

1. **{Cheese, Milk}** (count = 2)
   - Cheese $\implies$ Milk: $2/2 = 100\% \geq 75\% \rightarrow$ **Strong Rule**
   - Milk $\implies$ Cheese: $2/3 = 66.67\% < 75\% \rightarrow$ *Rejected*
2. **{Milk, Butter}** (count = 2)
   - Milk $\implies$ Butter: $2/3 = 66.67\% < 75\% \rightarrow$ *Rejected*
   - Butter $\implies$ Milk: $2/3 = 66.67\% < 75\% \rightarrow$ *Rejected*
3. **{Milk, Bread}** (count = 2)
   - Milk $\implies$ Bread: $2/3 = 66.67\% < 75\% \rightarrow$ *Rejected*
   - Bread $\implies$ Milk: $2/3 = 66.67\% < 75\% \rightarrow$ *Rejected*
4. **{Butter, Bread}** (count = 3)
   - Butter $\implies$ Bread: $3/3 = 100\% \geq 75\% \rightarrow$ **Strong Rule**
   - Bread $\implies$ Butter: $3/3 = 100\% \geq 75\% \rightarrow$ **Strong Rule**

From L_3: {Milk, Butter, Bread} (count = 2)

- {Milk, Butter} $\implies$ Bread: $2/2 = 100\% \geq 75\% \rightarrow$ **Strong Rule**
- {Milk, Bread} $\implies$ Butter: $2/2 = 100\% \geq 75\% \rightarrow$ **Strong Rule**
- {Butter, Bread} $\implies$ Milk: $2/3 = 66.67\% < 75\% \rightarrow$ *Rejected*
- Milk $\implies$ {Butter, Bread}: $2/3 = 66.67\% < 75\% \rightarrow$ *Rejected*
- Butter $\implies$ {Milk, Bread}: $2/3 = 66.67\% < 75\% \rightarrow$ *Rejected*
- Bread $\implies$ {Milk, Butter}: $2/3 = 66.67\% < 75\% \rightarrow$ *Rejected*

Final Strong Association Rules ($\text{Confidence} \geq 75\%$):

1. **Cheese** $\implies$ **Milk** (Support = 50%, Confidence = 100%)
2. **Butter** $\implies$ **Bread** (Support = 75%, Confidence = 100%)
3. **Bread** $\implies$ **Butter** (Support = 75%, Confidence = 100%)
4. **{Milk, Butter}** $\implies$ **Bread** (Support = 50%, Confidence = 100%)
5. **{Milk, Bread}** $\implies$ **Butter** (Support = 50%, Confidence = 100%)

Response: 5. a) Compare association and classification. [2 Marks]

Feature	Association Rule Mining	Classification
Learning Type	**Unsupervised learning** (no target class label required).	**Supervised learning** (requires predefined class labels).
Objective	Discovers interesting relationships, correlations, or co-occurrences among items ($X \implies Y$).	Predicts the discrete class label or category of an unknown instance based on training data.
Target Variable	Any attribute can appear on the left-hand side (antecedent) or right-hand side (consequent).	The right-hand side is strictly restricted to a single predetermined **target/class attribute**.
Evaluation Metrics	**Support** and **Confidence** (also Lift).	**Accuracy**, **Precision**, **Recall**, and **F1-Score**.

5. a) (continued) Briefly explain associative classification with suitable example. [3 Marks]

Definition:

Associative Classification is an integrated data mining approach that uses association rule mining techniques to build a classification model. It searches for strong association rules where the right-hand side (consequent) is strictly constrained to a **class label**. These rules are known as **Class Association Rules (CARs)**.

$$\text{Format: Condition (Attribute-Value Pairs)} \implies \text{Class Label}$$

Working Process:

1. Mine all frequent and high-confidence CARs meeting minimum support and confidence thresholds.
2. Prune redundant or low-quality rules and rank them (e.g., CBA – Classification Based on Associations).
3. Use the ranked rules as a classifier to assign class labels to incoming unseen instances.

Example:

In a banking credit risk dataset:

- Normal Association Rule:

$$\text{Age} \leq 30 \implies \text{Buys_Laptop} \quad [\text{Support} = 20\%, \text{Confidence} = 80\%]$$

- **Class Association Rule (Associative Classification):**

$$\text{Income} = \text{"Low"} \land \text{Credit_Score} = \text{"Poor"} \implies \text{Loan_Status} = \text{"Rejected"} \quad [\text{Support} = 15\%, \text{Confidence} = 92\%]$$

(Here, the consequent is strictly the target class label `Loan_Status`).

5. b) What is an attribute? How is the Attribute-Relation File Format file constructed? [3 Marks]

1. What is an Attribute?

An **attribute** (also known as a field, feature, or variable) is a distinct data dimension that represents a characteristic, property, or measurement of a data object/tuple (e.g., `Age`, `Salary`, `Blood_Type`).

2. Construction of an Attribute-Relation File Format (ARFF) File

An **ARFF** file is an ASCII text file format developed for the Weka machine learning software. It consists of two primary sections:

1. **Header Section:**
   - `@relation <relation-name>`: Defines the dataset name.
   - `@attribute <attribute-name> <datatype>`: Declares each attribute and its data type (e.g., `NUMERIC`, `STRING`, `DATE`, or nominal values listed in `{...}`).
2. **Data Section:**
   - `@data`: Denotes the start of the data records.
   - Followed by comma-separated lines of values corresponding to the attributes in the header.

Example Structure:

```
@relation customer_data

@attribute age numeric
@attribute income {low, medium, high}
@attribute buys_computer {yes, no}

@data
25, medium, yes
45, high, yes
35, low, no
```

5. c) How is the IF – THEN logic achieved? Provide an example. [6 Marks]

1. Representation of IF – THEN Logic

In data mining, an **IF – THEN rule** takes the form:

$$\textbf{IF } \text{condition } \textbf{THEN } \text{conclusion}$$

- **IF (Antecedent / Precondition):** A conjunction of attribute tests connected by logical `AND` operators:

$$(A_1 = v_1) \land (A_2 = v_2) \land \cdots \land (A_k = v_k)$$

- **THEN (Consequent):** The predicted target class label:

$$(C = c_j)$$

2. How IF – THEN Logic is Achieved / Extracted

There are two primary ways to generate IF – THEN rules:

1. **Extraction from Decision Trees (Rule-Based Extraction):**
   - A decision tree is built using algorithms like ID3, C4.5, or CART.
   - Each path from the **root node** to a **leaf node** is directly translated into one distinct IF – THEN rule.
   - The split conditions along the branches form the `IF` condition, and the class label at the leaf forms the `THEN` conclusion.

- Rules extracted from a single tree are mutually exclusive and exhaustive.

2. **Sequential Covering Algorithms (Direct Rule Induction):**
   - Algorithms like **RIPPER**, **CN2**, or **PRISM** extract rules directly from training data without creating a decision tree.
   - It learns one rule at a time:
     1. Starts with an empty rule and greedily adds attribute conditions that maximize class purity.
     2. Once a rule is formed, all training instances covered by this rule are removed ("covered").
     3. The process repeats iteratively on the remaining dataset until all tuples are classified.
3. **Rule Quality Measures:**
   - **Coverage:** Fraction of tuples in dataset D that satisfy the antecedent:

$$\text{Coverage}(R) = \frac{n_{\text{covers}}}{|D|}$$

   - **Accuracy:** Fraction of covered tuples where the predicted class is correct:

$$\text{Accuracy}(R) = \frac{n_{\text{correct}}}{n_{\text{covers}}}$$

3. Concrete Example

Consider a loan approval dataset with features `Credit_Score`, `Annual_Income`, and class label `Loan_Approved`:

- **Decision Tree Path:**

  Root: `Credit_Score > 700`

  $\rightarrow$ True branch: `Annual_Income > $50,000` $\rightarrow$ Leaf: `Loan_Approved = Yes`

- **Extracted IF – THEN Rule:**

$$\textbf{IF } (\text{Credit_Score} > 700) \wedge (\text{Annual_Income} > \$50,000) \textbf{ THEN } \text{Loan_Approved} = \text{Yes}$$

- **Evaluation:**
  - If 100 records satisfy this condition out of 1000 total records, $\text{Coverage} = \frac{100}{1000} = 10\%$.
  - If 95 of those 100 applicants actually had their loans approved, $\text{Accuracy} = \frac{95}{100} = 95\%$.

Response: Here are the complete, structured exam-ready answers for **Questions 6, 7, and 8** based on **Table – 01**.

Question 6

6. a) What are the different item sets in association learning? [3 Marks]

In association rule mining, an **itemset** is a collection of one or more items. The primary categories are:

1. **k-Itemset:** An itemset containing exactly k items (e.g., $\{\text{Sunny}, \text{Play=No}\}$ is a 2-itemset).
2. **Frequent (Large) Itemset:** An itemset whose occurrence frequency (support count) satisfies the user-defined minimum support threshold:

$$\text{Support}(I) \geq \text{min_sup}$$

3. **Closed Itemset:** An itemset X where none of its immediate supersets has the same support count as X.
4. **Maximal Itemset:** A frequent itemset that has no frequent proper superset.

6. b) Refer to Table – 01: How many one-item and four-item sets can be prepared? [8 Marks]

1. Total Number of Distinct Items (m)

To form itemsets from a relational table, each unique **(Attribute = Value)** pair is treated as a distinct item.

- **Outlook (3 values):** `Sunny`, `Overcast`, `Rainy`
- **Temperature (12 distinct values):** `85`, `80`, `83`, `70`, `68`, `65`, `64`, `72`, `69`, `75`, `81`, `71`
- **Humidity (10 distinct values):** `85`, `90`, `86`, `96`, `80`, `70`, `65`, `95`, `75`, `91`
- **Windy (2 values):** `FALSE`, `TRUE`
- **Play (2 values):** `No`, `Yes`

$$\text{Total Distinct Items } (m) = 3 + 12 + 10 + 2 + 2 = 29$$

2. Number of 1-Item Sets

A 1-itemset contains exactly 1 item chosen from the available pool:

$$\text{Total 1-item sets} = \binom{29}{1} = \mathbf{29}$$

3. Number of 4-Item Sets

The number of 4-itemsets can be answered under two standard exam interpretations:

- **Interpretation A: Unconstrained Combinatorial Selection**

  Choosing any 4 items out of the total 29 distinct items:

  $$\binom{29}{4} = \frac{29 \times 28 \times 27 \times 26}{4 \times 3 \times 2 \times 1} = \mathbf{23,751}$$

- **Interpretation B: Meaningful Multi-Attribute Itemsets (1 value per distinct attribute)**

  A transaction cannot take two values for the same attribute (e.g., an instance cannot be both `Sunny` and `Rainy`). Thus, choosing 4 items across 4 distinct attributes out of the 5 available columns:

  - Choose 4 attributes out of 5:
    1. **Outlook** $\times$ **Temp** $\times$ **Humidity** $\times$ **Windy:** $3 \times 12 \times 10 \times 2 = 720$
    2. **Outlook** $\times$ **Temp** $\times$ **Humidity** $\times$ **Play:** $3 \times 12 \times 10 \times 2 = 720$
    3. **Outlook** $\times$ **Temp** $\times$ **Windy** $\times$ **Play:** $3 \times 12 \times 2 \times 2 = 144$
    4. **Outlook** $\times$ **Humidity** $\times$ **Windy** $\times$ **Play:** $3 \times 10 \times 2 \times 2 = 120$
    5. **Temp** $\times$ **Humidity** $\times$ **Windy** $\times$ **Play:** $12 \times 10 \times 2 \times 2 = 480$

  $$\text{Total Meaningful 4-item sets} = 720 + 720 + 144 + 120 + 480 = \mathbf{2,184}$$

(Writing both interpretations ensures maximum marks).

6. c) Why is the decision set important in data mining? Give one example. [3 Marks]

- **Importance:** A **decision set** (or rule set) consists of an unordered or ordered collection of mutually independent `IF-THEN` rules. It is vital because:
  1. **High Interpretability:** Each rule is self-contained and human-readable, unlike complex black-box models.
  2. **Modularity & Scalability:** Rules can be updated, inserted, or pruned independently without retraining an entire decision network.
  3. **Robustness:** Handles overlapping boundaries and edge cases gracefully.
- **Example:**

$$\textbf{IF } \text{Outlook} = \text{Overcast } \textbf{THEN } \text{Play} = \text{Yes}$$

$$\textbf{IF } \text{Humidity} > 75 \land \text{Windy} = \text{TRUE } \textbf{THEN } \text{Play} = \text{No}$$

Question 7

7. a) What is Decision Tree? Explain how classification is done using decision tree induction. [7 Marks]

1. What is a Decision Tree?

A **Decision Tree** is a supervised learning model structured like a flowchart:

- **Root Node:** Represents the initial feature from which data splitting begins.
- **Internal (Decision) Nodes:** Represent conditional tests on specific attributes.
- **Branches:** Represent the outcomes of the tests.
- **Leaf Nodes:** Represent terminal class labels or target predictions.

2. Classification Process via Decision Tree Induction

1. **Tree Construction (Induction / Training Phase):**
    - Built top-down via a greedy recursive divide-and-conquer strategy.
    - At each node, an attribute selection measure (such as Information Gain, Gain Ratio, or Gini Index) evaluates all candidate splits.
    - The attribute that yields the highest purity / lowest impurity is selected to split the dataset.
    - This partitioning repeats recursively on each branch until a stopping criterion is met:
        - All samples at a node belong to the same class.
        - No remaining attributes are available for further splitting.
        - Maximum tree depth or minimum leaf sample count is reached.
2. **Pruning Phase:**
    - Removes unnecessary branches to reduce complexity and prevent overfitting.
3. **Inference / Prediction Phase:**
    - An unknown test tuple starts at the root node.
    - It traverses branches by matching its feature values against the conditions at each internal node until it reaches a leaf node, which assigns the predicted class label.

7. b) Generate decision tree using CART algorithm using the table – 01. [7 Marks]

The **CART (Classification and Regression Trees)** algorithm uses the **Gini Index** to create strictly **binary splits.**

Step 1: Overall Dataset Impurity (D)

Total instances $N = 14$:

- $\text{Play} = \text{Yes} : 9$
- $\text{Play} = \text{No} : 5$

$$\text{Gini}(D) = 1 - \left[\left(\frac{9}{14} \right)^2 + \left(\frac{5}{14} \right)^2 \right] = 1 - [0.4133 + 0.1276] = \mathbf{0.459}$$

Step 2: Evaluating Potential Root Splits using $\text{Gini}_A(D)$

- **Attribute: Windy** (Values: `TRUE`, `FALSE`)
    - `Windy = TRUE` (6 instances: 3 Yes, 3 No):

$$\text{Gini}(\text{Windy}_{\text{TRUE}}) = 1 - \left[\left(\frac{3}{6} \right)^2 + \left(\frac{3}{6} \right)^2 \right] = 0.500$$

- `Windy = FALSE` (8 instances: 6 Yes, 2 No):

$$\text{Gini}(\text{Windy}_{\text{FALSE}}) = 1 - \left[\left(\frac{6}{8}\right)^2 + \left(\frac{2}{8}\right)^2\right] = 1 - [0.5625 + 0.0625] = 0.375$$

- **Weighted Gini:**

$$\text{Gini}_{\text{Windy}}(D) = \frac{6}{14}(0.500) + \frac{8}{14}(0.375) = 0.214 + 0.214 = \mathbf{0.428}$$

- **Attribute: Outlook** (Binary candidate split: `{Overcast}` vs `{Sunny, Rainy}`)
  - `Outlook = Overcast` (4 instances: 4 Yes, 0 No) $\rightarrow$ **Pure Leaf!**

$$\text{Gini}(\text{Overcast}) = 1 - \left[\left(\frac{4}{4}\right)^2 + 0\right] = 0$$

  - `Outlook } \in {\text{Sunny, Rainy}}$ (10 instances: 5 Yes, 5 No):

$$\text{Gini}(\text{Sunny, Rainy}) = 1 - \left[\left(\frac{5}{10}\right)^2 + \left(\frac{5}{10}\right)^2\right] = 0.500$$

  - **Weighted Gini:**

$$\text{Gini}_{\text{Outlook}=\{\text{Overcast}\}}(D) = \frac{4}{14}(0) + \frac{10}{14}(0.500) = \mathbf{0.357}$$

- **Attribute: Humidity** (Standard binary split at threshold ≤ 75 vs > 75):
  - ≤ 75 (6 instances: 5 Yes, 1 No) $\rightarrow \text{Gini} = 1 - [(5/6)^2 + (1/6)^2] = 0.278$
  - > 75 (8 instances: 4 Yes, 4 No) $\rightarrow \text{Gini} = 0.500$
  - $\text{Gini}_{\text{Humidity}}(D) = \frac{6}{14}(0.278) + \frac{8}{14}(0.500) = 0.119 + 0.286 = \mathbf{0.405}$

  Root Node Selection:

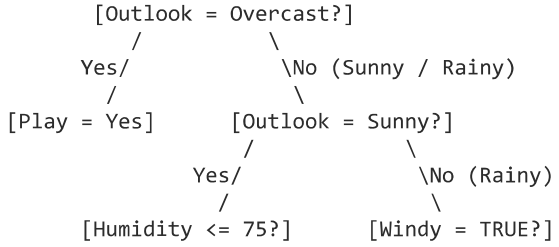
  The split $\text{Outlook} = \text{Overcast}$ **vs** $\{\text{Sunny, Rainy}\}$ produces the lowest Gini index ($\mathbf{0.357}$) and is selected as the **Root Node.**

Step 3: Splitting the Non-Pure Branch ($\{\text{Sunny, Rainy}\}$)

For the remaining 10 instances (5 Sunny, 5 Rainy):

- On the **Sunny** subset (5 instances: 2 Yes, 3 No), split on **Humidity** ≤ 75:
  - Humidity $\leq 75 \implies \text{Play} = \text{Yes}$ (Pure: 2 Yes, 0 No)
  - Humidity $> 75 \implies \text{Play} = \text{No}$ (Pure: 0 Yes, 3 No)
- On the **Rainy** subset (5 instances: 3 Yes, 2 No), split on **Windy**:
  - Windy = FALSE $\implies \text{Play} = \text{Yes}$ (Pure: 3 Yes, 0 No)
  - Windy = TRUE $\implies \text{Play} = \text{No}$ (Pure: 0 Yes, 2 No)

Resulting CART Binary Decision Tree:



```
        /         \           /         \
     Yes/           \No    Yes/           \No
      /             \       /             \
[Play=Yes]     [Play=No] [Play=No]     [Play=Yes]
```

Question 8

8. a) How is clustering accomplished? [2 Marks]

Clustering is an **unsupervised learning process** accomplished through the following core steps:

1. **Feature Representation & Scaling:** Select relevant numerical or categorical attributes and normalize/standardize them.
2. **Proximity / Distance Measure:** Define a similarity or distance metric (e.g., Euclidean distance, Manhattan distance, or Cosine similarity).
3. **Partitioning / Grouping Algorithm:** Apply an algorithm (e.g., **K-Means**, **Hierarchical clustering**, or **DBSCAN**) that iteratively groups data objects to maximize intra-cluster similarity and maximize inter-cluster separation.
4. **Validation & Assessment:** Evaluate clusters using metrics such as the silhouette coefficient or sum of squared errors (SSE).

8. b) How many clusters are possible for the table – 01? [6 Marks]

In unsupervised cluster analysis, the number of clusters K can be determined depending on the perspective and constraints:

1. **Theoretical Mathematical Range:** For a dataset with $N = 14$ tuples:
   - **Minimum possible clusters:** $K_{\min} = 1$ (all 14 instances belong to one single cluster).
   - **Maximum possible clusters:** $K_{\max} = 14$ (each individual instance forms its own cluster).
   - **Range:** $1 \leq K \leq 14$
2. **Empirical / Rule of Thumb:** Using the standard statistical rule of thumb:

$$K \approx \sqrt{\frac{N}{2}} = \sqrt{\frac{14}{2}} = \sqrt{7} \approx \mathbf{2.65} \implies \mathbf{2} \text{ or } \mathbf{3} \text{ clusters}$$

3. **Natural Domain-Based Clusters:**
   - **Target-Based Clustering:** $K = 2$ clusters (corresponding naturally to `Play = Yes` and `Play = No`).
   - **Environmental/Outlook-Based Clustering:** $K = 3$ clusters (corresponding to weather conditions: `Sunny`, `Overcast`, and `Rainy`).

8. c) How many nodes are possible for Table – 01, if you consider a decision tree? [6 Marks]

The exact count of nodes depends on tree depth and stopping constraints:

1. **Theoretical Bounds:**
   - **Minimum Nodes (1 node):** A trivial tree with only **1 root node** (all instances classified into the majority class `Play = Yes` without any splits).
   - **Maximum Nodes (Fully Grown/Overfit Tree):**

     Since there are $N = 14$ instances, a maximally shattered decision tree can have at most 14 pure leaves. For any binary tree with L leaves, the number of internal nodes is $L - 1$.

$$\text{Total Nodes} = 2L - 1 = 2(14) - 1 = \mathbf{27} \text{ nodes}$$

2. **Practical / Optimal CART Tree (from Question 7b):**
   - **Internal Decision Nodes (4):**
     1. `Outlook = Overcast?`
     2. `Outlook = Sunny?`
     3. `Humidity <= 75?`

4. `Windy = TRUE?`

- **Leaf / Terminal Nodes (5):**
  1. Leaf 1: `Play = Yes` (Overcast)
  2. Leaf 2: `Play = Yes` (Sunny, Low Humidity)
  3. Leaf 3: `Play = No` (Sunny, High Humidity)
  4. Leaf 4: `Play = No` (Rainy, Windy)
  5. Leaf 5: `Play = Yes` (Rainy, Not Windy)

$$\text{Total Practical Nodes} = 4\ (\text{Internal}) + 5\ (\text{Leaves}) = \mathbf{9}\ \text{nodes}$$
